Technical Case (NestJS, MongoDB)

1. Main Problem

As an estate agency consultancy, one of the most critical moments for us is the process that begins **after an agreement is reached** for the sale or rental of a property.

Currently, this process (earnest money, title deeds, payments, etc.) is largely manual and tracked using various tools. Once the transaction is completed, the distribution of the total service fee (commission) between the company and the agents is both time-consuming and prone to human error due to complex rules.

We need a backend system that:

- Automates this process
- Ensures traceability
- Accurately manages the value flow for all financial stakeholders

---

2. Expected Solution

We expect you to design and develop the **core backend logic** of a system that solves the problem described above, using the **required tech stack**.

The system should have the following capabilities:

- Track the **lifecycle of a transaction** from start to finish
- Automatically **distribute the total service fee** to agency and agents
- Provide a **clear financial breakdown** for any transaction
- Expose APIs that allow interaction with the system

You are free to decide:

- How to structure your NestJS modules, services, controllers
- How to organize business logic
- How to design your data models
- How to structure your API endpoints

---

3. Tech Stack Requirements (Mandatory)

To ensure consistency across submissions, the following stack is **required**:

Language & Runtime

- Node.js (LTS)
- TypeScript

Framework

- **NestJS** (mandatory)

Database

- **MongoDB Atlas** (mandatory)
  Local MongoDB or self-hosted MongoDB instances are *not accepted*.

Database Access

- Mongoose **or** MongoDB Node.js driver

Testing

- Jest (recommended; NestJS default)

Submissions using other frameworks or databases will **not be evaluated**.

---

4. Core Rules and Scenarios

4.1 Transaction Stages

A transaction goes through specific stages:

- `agreement`
- `earnest_money`
- `title_deed`
- `completed`

Your system should:

- Track the stage of every transaction
- Allow stage transitions
- Optionally prevent invalid transitions (your decision — explain in DESIGN.md)

4.2 Financial Breakdown

The system should clearly report **who earned how much** for each completed transaction:

- How much the **agency** earned
- How much each **agent** earned
- Why they earned that amount (listing agent / selling agent)

You may store this breakdown:

- Embedded in the transaction document,

- In a dedicated collection,
- Or compute it dynamically.

Your DESIGN.md should justify your approach.

4.3 Company Commission Policy

- **50%** of the total service fee belongs to the **agency**
- The remaining **50%** is shared among the **agents involved**

Scenario 1

If the **listing agent** and the **selling agent** are **the same person**, that agent receives **100% of the agent portion (50%)**.

Scenario 2

If the **listing agent** and **selling agent** are **different**, they share the agent portion **equally** (25% each).

Your code and tests must implement these rules.

---

5. Design Freedom and Responsibilities

Within the constraints of **NestJS + MongoDB Atlas**, you are free to design:

- Module organization
- Service structure
- Database schema
- Business logic separation
- API endpoints

- Error handling
- Validation structure (DTOs, pipes, etc.)

Your design choices and reasoning must be explained in **DESIGN.md**.

---

6. Deliverables

6.1 Source Code (Git repository)

Provide a link to a public Git repository containing your working NestJS project.

Repository must include:

- `src/` folder
- NestJS modules, controllers, services
- MongoDB connection setup
- Test folder (if separated)

6.2 README.md

Must include:

- Installation instructions
- How to configure environment variables (including MongoDB Atlas)
- How to run the project
- How to run tests
- **Live API URL** from deployment

Docker is optional.

6.3 Unit Tests (Mandatory)

Tests should validate:

- Commission calculation rules
- Transaction stage transitions
- Core business logic

Focus on service-level tests; e2e tests are optional.

6.4 DESIGN.md (Mandatory)

Explain the following:

1. **Architecture & Data Model**
    - How you structured your NestJS modules and layers
    - How you designed MongoDB documents/collections
    - Why you chose this structure
    - What alternatives you rejected and why
2. **Most Challenging / Riskiest Part**
    - What design decision was risky?
    - How did you mitigate that risk?
3. **If Implemented in Real Life — What Next?**
    - What would be the next features or improvements?
    - Why those? (e.g., auditing, rule engine, auth, reporting)

Include diagrams if helpful.

6.5 Deployment (Mandatory)

You must **deploy** your backend to a free-tier hosting platform.

You may use:

- **Vercel**
- **Render**
- **Railway**
- **Fly.io**

Database Requirement

Your deployment **must use MongoDB Atlas**.
Local MongoDB or self-hosted DB is *not accepted*.

Deployment Requirements

Your deployment must include:

- A **public API URL**
- NestJS running with **MongoDB Atlas**
- At least **one functional endpoint** (e.g. `/health`, `/transactions`)

README Must Include:

1. Live API URL
2. Local development instructions
3. Environment variables (including MongoDB Atlas connection string)

If deployment is not completed, you must explain why —
but this will **affect evaluation**.

---

7. Evaluation Criteria

We evaluate based on:

✔ Problem Analysis & Data Modeling

Did you properly understand the domain and model it correctly?

✔ System Design & Architecture

Are your modules, services, and layers clear and extensible?

✔ Code Quality

Is your code readable, structured, validated, and testable?

✔ Business Logic Implementation

Are the commission rules correctly implemented?

✔ Testing

Do tests verify the core business rules?

✔ Deployment

Is the backend actually live and functioning?

✔ Decision Justification

Are your design decisions properly explained in DESIGN.md?

---

8. Deadline

You may choose your own deadline. However, both the **time you take** and the **quality of results** will influence our evaluation.

Good luck!

This task is more than a coding exercise — it's an **engineering and design challenge.** You have full autonomy within the **NestJS + MongoDB Atlas** constraints.